

Checklist

Authentication

Identity and Access Management (IAM)

- ☐ Understand the purpose of **Identity and Access Management (IAM)**.
- ☐ Apply the **principle of least privilege**.
- ☐ Centralize authentication and account management when practical.
- ☐ Understand how **Single Sign-On (SSO)** reduces password sprawl.
- ☐ Know the role of **SCIM** in automated user provisioning and deprovisioning.
- ☐ Remove unnecessary, obsolete, or unused accounts and assets.
- ☐ Disable access promptly when employees or vendors leave.
- ☐ Combine strong password management with **MFA**.
- ☐ Avoid relying on a single security control.

Password Security

- ☐ Use **long, unique passwords or passphrases**.
- ☐ Avoid short passwords even when they contain mixed character types.
- ☐ Understand that password length greatly increases brute-force difficulty.
- ☐ Avoid predictable substitutions and personal information.
- ☐ Consider randomly generated passphrases, such as **Diceware**.
- ☐ Never share passwords with coworkers or help-desk staff.
- ☐ Train users to report anyone requesting their password.
- ☐ Never reuse passwords across different accounts or services.
- ☐ Assume exposed credentials may be tested against other services through **credential stuffing**.
- ☐ Avoid storing passwords in plain text.

Password Attacks

- ☐ Understand **brute-force attacks**.
- ☐ Understand **dictionary attacks**.

- ☐ Understand **rainbow-table attacks**.
- ☐ Know why salts reduce the usefulness of precomputed rainbow tables.
- ☐ Recognize the danger of compromised credential databases.
- ☐ Know that weak passwords remain vulnerable even when stored as hashes.

Encryption

- ☐ Understand that **encryption is reversible** when the correct key is available.
- ☐ Know that encryption protects data **in transit** and **at rest**.
- ☐ Recognize commonly used modern algorithms such as **AES**, **RSA**, and **ECC**.
- ☐ Understand that advances in quantum computing create risks for some current cryptographic systems.
- ☐ Be familiar with the purpose of **post-quantum/quantum-resistant cryptography**.
- ☐ Avoid deprecated encryption algorithms.

Hashing

- ☐ Understand that **hashing is a one-way process**.
- ☐ Know that hashes produce a fixed-length output.
- ☐ Understand that even a small input change should produce a significantly different hash.
- ☐ Know that hash collisions are theoretically possible.
- ☐ Store password hashes instead of plaintext passwords.
- ☐ Use modern password-hashing or key-derivation functions rather than fast general-purpose hashes.

Salting

- ☐ Understand that a **salt** is extra data added before hashing.
- ☐ Use a different random salt for each stored password.
- ☐ Know that salting prevents identical passwords from producing identical stored hashes.
- ☐ Understand how salting makes precomputed attacks more difficult.
- ☐ Recognize **bcrypt** as an example of a password-hashing algorithm that uses salts.

Cryptographic Algorithm Hygiene

- ☐ Replace obsolete algorithms with stronger alternatives.
- ☐ Avoid **MD5**.
- ☐ Avoid **SHA-1**.
- ☐ Avoid **DES/3DES**.
- ☐ Avoid weak RSA/DSA key sizes.
- ☐ Prefer modern, currently supported cryptographic standards.
- ☐ Follow current **NIST** recommendations for password hashing and key derivation.
- ☐ Use sufficiently large salts and high iteration/work factors for password hashing.

Password Managers

- ☐ Use a password manager to reduce password reuse.
- ☐ Protect the password manager with a strong master password.
- ☐ Enable **MFA** for the password manager.
- ☐ Determine whether passwords must be shared among administrators or teams.
- ☐ Check whether the manager supports password generation.
- ☐ Check whether it includes password-strength evaluation.
- ☐ Check whether it supports autofill where appropriate.
- ☐ Verify that stored passwords are strongly encrypted.
- ☐ Check for an automatic lockout feature.
- ☐ Review protection against malicious activity such as keylogging.
- ☐ Review available auditing and logging features.
- ☐ Obtain software directly from the vendor.
- ☐ Verify installer integrity when hashes or signatures are provided.

Password Reset Security

- ☐ Avoid weak knowledge-based security questions.
- ☐ Do not use answers that can easily be found online or guessed.
- ☐ Use randomized answers when security questions cannot be avoided.
- ☐ Store those randomized answers in a password manager.
- ☐ Protect password-reset processes with MFA when possible.

Password Storage and Infrastructure

- ☐ Review where passwords and credentials are stored.
- ☐ Avoid default passwords on servers and management interfaces.
- ☐ Change default credentials immediately.
- ☐ Protect management interfaces such as **BMC/iLO/IPMI**.
- ☐ Segment management interfaces onto restricted networks/VLANs.
- ☐ Use centralized authentication where possible instead of maintaining many local passwords.

SMB and Windows Authentication Hardening

- ☐ Require **SMB signing**.
- ☐ Enable SMB encryption where appropriate.
- ☐ Require modern SMB versions.
- ☐ Disable **SMBv1**.
- ☐ Prevent fallback to older, weaker SMB dialects.
- ☐ Disable insecure guest authentication.
- ☐ Review current Microsoft hardening guidance.

Fine-Grained Password Policies

- ☐ Understand the purpose of **Fine-Grained Password Policies (FGPPs)**.
- ☐ Use different password requirements for users or groups when justified.
- ☐ Verify that policies are applied to the correct users/security groups.
- ☐ Avoid unnecessary policy complexity.

Cloud IAM

- ☐ Evaluate cloud IAM features such as centralized authentication.
- ☐ Use banned-password lists where supported.
- ☐ Apply **conditional access policies**.
- ☐ Consider scalability, infrastructure cost, and remote-access benefits.
- ☐ Select cloud IAM based on requirements rather than assuming cloud automatically solves access-control problems.

Authentication Protocols

NTLM

- ☐ Know that **NTLM uses challenge-response authentication**.
- ☐ Understand the basic NTLM authentication flow.
- ☐ Know that the user's password hash is involved rather than sending the plaintext password directly.
- ☐ Recognize NTLM as a legacy authentication technology.
- ☐ Know common NTLM risks such as **pass-the-hash**.
- ☐ Prefer more secure authentication methods when supported.

Kerberos

- ☐ Know that **Kerberos uses tickets and third-party authentication**.
- ☐ Understand the role of the **Key Distribution Center (KDC)**.
- ☐ Know that the KDC contains the **Authentication Server (AS)** and **Ticket Granting Server (TGS)**.
- ☐ Understand the purpose of a **Ticket Granting Ticket (TGT)**.
- ☐ Understand the purpose of a service ticket.
- ☐ Know the basic Kerberos flow:
 - ☐ Client requests a TGT.
 - ☐ KDC verifies the client and returns the TGT and session key.
 - ☐ Client presents the TGT to the TGS.
 - ☐ TGS returns a service ticket/session key.
 - ☐ Client presents the service ticket to the resource server.
- ☐ Recognize attacks such as **pass-the-ticket**.
- ☐ Recognize **Golden Ticket** attacks.
- ☐ Recognize **Silver Ticket** attacks.
- ☐ Recognize **Kerberoasting**.
- ☐ Recognize **AS-REP roasting**.
- ☐ Avoid weak/deprecated Kerberos encryption where possible.

LDAP

- ☐ Understand that **LDAP** provides directory access and can be used for authentication.
- ☐ Understand a basic **LDAP bind**.
- ☐ Know that LDAP can expose credentials if used without transport protection.
- ☐ Prefer **LDAPS/TLS** rather than plaintext LDAP authentication.

- ☐ Recognize common LDAP risks such as:
 - ☐ Account discovery.
 - ☐ Brute-force attacks.
 - ☐ Domain enumeration.

RADIUS

- ☐ Understand that **RADIUS** uses a client/server authentication model.
- ☐ Know the role of a **NAS/RAS** as an intermediary.
- ☐ Understand **Access-Request** messages.
- ☐ Understand **Access-Challenge** messages.
- ☐ Understand **Access-Accept** messages.
- ☐ Understand **Access-Reject** messages.
- ☐ Know that RADIUS can provide **authentication, authorization, and accounting (AAA)**.
- ☐ Understand start, update, and stop accounting messages.
- ☐ Recognize common authentication mechanisms such as **EAP-TLS**, **PEAP-MSCHAPv2**, and **EAP-TTLS/PAP**.
- ☐ Protect RADIUS shared secrets and authentication traffic.

OIDC and SAML

- ☐ Know that **OIDC** is token-based and built on OAuth 2.0 flows.
- ☐ Know that OIDC commonly relies on **TLS** for transport security.
- ☐ Recognize risks such as token interception and client impersonation.
- ☐ Know that **SAML** uses XML-based assertions.
- ☐ Recognize SAML risks such as replay attacks.
- ☐ Recognize signature-wrapping attacks.
- ☐ Recognize XML injection risks.
- ☐ Recognize assertion theft.
- ☐ Protect both protocols against man-in-the-middle attacks.

Protocol Security

- ☐ Verify which authentication protocol a system actually uses.
- ☐ Review authentication settings in configuration files, GUIs, or management interfaces.
- ☐ Check authentication logs for protocol usage.

- ☐ Use packet-analysis tools such as **Wireshark** when appropriate.
- ☐ Review vendor documentation for supported protocols.
- ☐ Avoid protocols that transmit credentials in plaintext.
- ☐ Disable insecure protocols such as **Telnet**, **FTP**, **SNMPv2**, **HTTP**, and old SMB versions whenever possible.

Choosing an Authentication Protocol

- ☐ Identify the application's authentication requirements.
- ☐ Determine whether the solution needs to scale.
- ☐ Evaluate management complexity and technical debt.
- ☐ Define security requirements.
- ☐ Evaluate performance constraints.
- ☐ Determine interoperability requirements.
- ☐ Perform **threat modeling** before production deployment.

Multi-Factor Authentication (MFA)

MFA Fundamentals

- ☐ Understand that MFA requires multiple independent authentication factors.
- ☐ Know the common factor categories:
 - ☐ Something you **know**.
 - ☐ Something you **have**.
 - ☐ Something you **are**.
- ☐ Use MFA in addition to strong passwords rather than as a replacement for other controls.
- ☐ Require MFA for privileged accounts.
- ☐ Require MFA for remote access.
- ☐ Require MFA for VPN access.
- ☐ Require MFA for email and portals.
- ☐ Extend MFA to employees, vendors, consultants, and other users where appropriate.

MFA Weaknesses

- ☐ Do not assume MFA eliminates phishing.
- ☐ Recognize **MFA fatigue/push bombing** attacks.
- ☐ Train users not to approve unexpected authentication prompts.
- ☐ Prefer methods that require entering or matching a code rather than blindly approving a push.
- ☐ Protect MFA enrollment and reset processes.
- ☐ Review systems that may have been excluded from MFA deployment.
- ☐ Avoid inconsistent MFA implementation across an organization.
- ☐ Understand that implementation quality determines how effective MFA is.

Final Review

- ☐ Can I explain the difference between **encryption, hashing, and salting**?
- ☐ Can I explain why long, unique passphrases are safer than short complex passwords?
- ☐ Can I explain how a password manager improves security?
- ☐ Can I describe the basic authentication flow of **NTLM**?
- ☐ Can I describe the basic ticket flow of **Kerberos**?
- ☐ Can I explain an **LDAP bind**?
- ☐ Can I explain the **RADIUS AAA process**?
- ☐ Can I compare **NTLM, Kerberos, LDAP, RADIUS, OIDC, and SAML**?
- ☐ Can I identify deprecated or insecure protocols and algorithms?
- ☐ Can I explain why MFA improves security but does not eliminate all authentication attacks?
- ☐ Can I identify where MFA should be deployed first in an organization?
- ☐ Can I explain how least privilege, centralized IAM, strong passwords, secure protocols, and MFA work together as a layered defense?